Singable Lyrics Translation with IPA-Augmented LLM Agents

Anonymous ACL submission

Abstract

Translating song lyrics while preserving singability requires satisfying musical constraints, including syllable counts, rhyme schemes, and syllable patterns, that standard machine translation ignores. We present a framework that equips a large language model with IPA-based phonetic analysis tools within a ReAct-style agent loop. The agent extracts constraints from source lyrics, then translates through a multi-phase process, calling tools to iteratively verify and refine each constraint. We formalize the three musical constraints, describe the IPA tool suite and agent architecture, and propose three evaluation metrics: Syllable Error Rate (SER), Syllable Count Relative Error (SCRE), and Adjusted Rand Index (ARI). A phase ablation study confirms that iterative tool-verified refinement dramatically improves constraint satisfaction and preserves rhyme structure substantially better than a supervised baseline, while requiring no task-specific training, no parallel corpora, and generalizing to any of the 100+ languages supported by eSpeak-NG.

1 Introduction

A translated song lyric must conform to the musical structure of the original so that it remains *singable* to the same melody (Low, 2005). This requires satisfying hard structural constraints: syllable counts must match melodic phrases, rhyme schemes should be preserved, and word-level syllable distributions should mirror the original phrasing. Despite theoretical grounding (Low, 2005; Franzon, 2008), computational approaches remain scarce. Neural MT systems have no mechanism for phonetic constraints, and constrained decoding methods (Hokamp and Liu, 2017; Post and Vilar, 2018) address lexical but not phonetic constraints. A fundamental obstacle is that *counting syllables, detecting rhyme, and analyzing phonetic patterns are not tasks that LLMs perform reliably through text generation alone* (Gao et al., 2023).

We address this by equipping an LLM with IPA-based phonetic tools within a ReAct-style agent loop (Yao et al., 2023), enabling iterative, constraint-aware translation refinement. Unlike supervised approaches, the framework requires no parallel lyric corpora, supports any language pair via eSpeak-NG, and scales with LLM capability. Our framework is open-source and available as a PyPI package for reproducibility and practical use.[1]

Our contributions are:

1. A formalization of three musical constraints (syllable counts, rhyme schemes, and syllable patterns) and IPA-based methods for their extraction and verification (§3).

2. An IPA-augmented tool suite and multi-phase agent loop architecture for constraint-preserving lyrics translation (§4).

3. Three automatic evaluation metrics (SER, SCRE, and ARI) designed to measure structural constraint preservation in singable translation (§5).

To our knowledge, this is the first framework to combine IPA-based phonetic tools with LLM agents for singable lyrics translation.

2 Related Work

2.1 Song Translation

Low (2005) proposed treating song translation as a pentathlon balancing singability, sense, naturalness, rhythm, and rhyme, observing that rhythm is often the hardest constraint. Franzon (2008) identified five translation strategies ranging from adapting lyrics to rewriting both lyrics and melody. Low (2022) provided an updated theoretical account reinforcing singability as the guiding criterion.

[1] Code and package available at `https://github.com/anonymous` (anonymized for review).

On the computational side, Ou et al. (2023a) formalized singable lyric translation as a constrained sequence-to-sequence problem, fine-tuning a neural MT model with length, rhyme, and word boundary constraints for English–Chinese translation. Guo et al. (2022) addressed automatic song translation for tonal languages, introducing tonal constraints beyond syllable matching, and Ghazvininejad et al. (2018) tackled neural poetry translation with rhythm and rhyme constraints using constrained decoding. Ye et al. (2024) proposed quality-aware musical lyrics translation that explicitly models musical features during generation. Ren et al. (2025) introduced a curriculum reinforcement learning framework for controllable lyric translation, balancing multiple constraints through difficulty-aware training. Zhao et al. (2025) addressed melody-constrained lyrics editing, and Ou et al. (2023b) enforced structural wording and formatting constraints in melody-to-lyric generation. Cho et al. (2025) introduced MAVL, a multilingual audio-video lyrics dataset, Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation, and Kim et al. (2024) introduced a K-pop lyric translation dataset with Korean–English parallel data and neural modelling. Beyond translation, Chen and Teufel (2024) used scansion to generate Mandarin lyrics matching melodies, Liang et al. (2024) improved singability of AI-generated lyrics through prosody explanations, Yu et al. (2025) modeled lyric-melody alignment for generation, and Yoo et al. (2025) demonstrated that singability challenges extend to multimodal settings such as sign language. These approaches rely on specialized model fine-tuning, constrained decoding, or reinforcement learning; our work instead uses a general-purpose LLM augmented with IPA verification tools in an agent loop, requiring no task-specific training.

2.2 Constrained Text Generation

Hokamp and Liu (2017) and Post and Vilar (2018) introduced lexically constrained decoding via Grid Beam Search and Dynamic Beam Allocation. In computational poetry, Ghazvininejad et al. (2016) used finite-state machinery for meter and rhyme, Hopkins and Kiela (2017) generated rhythmic verse with neural networks, and Lau et al. (2018) jointly modeled poetic language, meter, and rhyme. These approaches enforce constraints at the decoding level or learn them implicitly; ours uses an agent loop where the LLM generates freely but verifies via external tools.

2.3 Tool-Augmented Language Models

Schick et al. (2023) trained language models to autonomously decide when to call tools such as calculators and search engines. ReAct (Yao et al., 2023) introduced a paradigm where LLMs alternate between generating reasoning traces and performing actions via external tools, improving performance on tasks requiring grounded, multi-step decision-making. PAL (Gao et al., 2023) demonstrated that offloading computational steps to program execution significantly improves accuracy on reasoning tasks, a finding directly relevant to our use case: syllable counting is a computational task that LLMs perform unreliably but that IPA-based tools handle deterministically. Chain-of-thought prompting (Wei et al., 2022) provides a complementary mechanism for structured reasoning that we combine with tool use in our agent loop. In machine translation, Jiao et al. (2023) demonstrated that LLMs achieve strong translation quality, but their evaluation did not consider musical or structural constraints.

2.4 Phonetic Resources

Our tool suite builds on Phonemizer (Bernard and Titeux, 2021) for text-to-IPA conversion and PanPhon (Mortensen et al., 2016) for phonetic distance computation; both are described in §4. Concurrently, Chen et al. (2025) explored using IPA as an intermediary representation for LLM-based translation of spoken-only languages; while their focus is on low-resource language preservation rather than musical constraints, both approaches share the insight that IPA provides a language-agnostic phonetic interface for LLMs.

3 Musical Constraints in Lyrics Translation

A singable translation must satisfy three structural constraints, each arising from a different aspect of the relationship between lyrics and melody.

3.1 Syllable Count Constraint

In sung performance, each syllable maps to one or more musical notes. When a melody assigns three notes to a phrase, the lyric must contain exactly three syllables; fewer leave notes unmatched, while extra syllables force cramming that disrupts the rhythmic feel. This makes syllable count the most rigid of the three constraints: rhyme and phrasing mismatches may be tolerable in some styles, but

syllable count mismatches are almost always audible. Formally, given source lyrics $L_s = (l_1, \ldots, l_n)$ in language λ_s, the constraint requires that translated lyrics $L_t = (l'_1, \ldots, l'_n)$ in target language λ_t satisfy $\text{syll}(l'_i, \lambda_t) = \text{syll}(l_i, \lambda_s)$ for all lines i.

Syllable counting is language-dependent. For Chinese, each character constitutes exactly one syllable, so counting is trivial. For other languages, we convert text to IPA via Phonemizer (Bernard and Titeux, 2021) and count *vowel nuclei*, the vocalic centers of syllables:

$$\text{syll}(l_i, \lambda_s) = \left|\text{nuclei}\big(\text{IPA}(l_i, \lambda_s)\big)\right| \quad (1)$$

where $\text{nuclei}(\cdot)$ matches a predefined set of IPA vowel symbols and diphthongs (e.g., a, i, u, ə, aɪ, eɪ, oʊ), with adjacent vowels forming diphthongs counted as single nuclei. For example, "*Let it go*" yields IPA /lɛt ɪt gəʊ/ with three vowel nuclei [ɛ, ɪ, əʊ], giving a count of 3. The resulting sequence $\mathbf{s} = (s_1, \ldots, s_n)$ serves as the hard constraint for translation.

3.2 Rhyme Scheme Constraint

Rhyme provides structural cohesion in lyrics, creating sonic expectations that are satisfying when met and jarring when violated. A song with an $AABB$ scheme feels fundamentally different from one with $ABAB$; preserving the rhyme scheme ensures the translation maintains the same pattern of sonic echoes as the original. Formally, a rhyme scheme is a labeling $R = (r_1, \ldots, r_n)$ where $r_i \in \{A, B, C, \ldots\}$ assigns each line to a rhyme class, with $r_i = r_j$ if and only if lines i and j rhyme. The constraint requires that the translation's scheme R_t match the source scheme R_s.

We detect rhyme by comparing phonetic endings rather than orthographic forms. For each line, we extract the *rhyme ending*: the IPA substring from the last vowel nucleus to the end of the transcription. For Chinese, we extract the pinyin final of the last character instead. The scheme is constructed by labeling the first line A, and assigning each subsequent line the label of any earlier line it rhymes with, or a new label otherwise. Consider the following example:

> *The stars shine bright with silver* ***light*** → /laɪt/ → ending: /aɪt/
> *And covers all in purest* ***white*** → /waɪt/ → ending: /aɪt/
> *I stand and watch the world* ***below*** → /bɪləʊ/ → ending: /əʊ/
> *As gentle winds begin to* ***blow*** → /bləʊ/ → ending: /əʊ/

Lines 1–2 share ending /aɪt/ (class A) and lines 3–4 share /əʊ/ (class B), yielding $AABB$. This IPA-based approach correctly identifies rhymes that orthographic comparison would miss (e.g., "weight" and "late") and rejects false rhymes based on spelling alone (e.g., "cough" and "through").

3.3 Syllable Pattern Constraint

Even when two lines share the same total syllable count, different word-level distributions create different rhythmic feels. Consider two eight-syllable lines: "un-for-get-ta-ble mel-o-dy" with pattern $[5, 3]$ versus "I can hear the mu-sic play" with pattern $[1, 1, 1, 1, 2, 2]$. The first has two long words producing a flowing feel, while the second has many short words producing a staccato rhythm. Word boundaries interact with note groupings and breaths in sung performance, so preserving the syllable pattern ensures the translation fits the melody at the phrasing level, not just the syllable level. Formally, the syllable pattern of a line l_i is a vector $\mathbf{p}_i = (p_{i,1}, \ldots, p_{i,k_i})$ recording the syllable count of each word. The constraint asks that the translation's pattern $\mathbf{p}'_i$ be similar to $\mathbf{p}_i$ while maintaining the same total: $\sum_j p'_{i,j} = \sum_j p_{i,j} = s_i$.

To extract patterns, we apply language-specific word segmentation (space-based tokenization for English and other space-delimited languages, and a neural tokenizer for Chinese), then compute each word's syllable count via Equation 1. We measure pattern similarity using a normalized alignment score. Given patterns of potentially different lengths, we pad the shorter one with zeros and compute:

$$\text{sim}(\mathbf{p}, \mathbf{p}') = 1 - \frac{\sum_{j=1}^{m} |p_j - p'_j|}{\sum_{j=1}^{m} \max(p_j, p'_j)} \quad (2)$$

where $m = \max(|\mathbf{p}|, |\mathbf{p}'|)$. A score of 1.0 indicates an exact match; we treat ≥ 0.8 as acceptable. For example, "Let it go" has pattern $[1, 1, 1]$. A Chinese translation tokenized as two words with pattern $[2, 1]$ yields $\text{sim}([1, 1, 1], [2, 1, 0]) = 0.5$ (poor alignment), whereas a translation tokenized as three single-character words gives pattern $[1, 1, 1]$ with $\text{sim} = 1.0$.

4 IPA-Augmented Agent Framework

Figure 1 overviews the agent architecture.

4.1 IPA as Universal Phonetic Interface

The International Phonetic Alphabet provides a standardized, language-independent representation

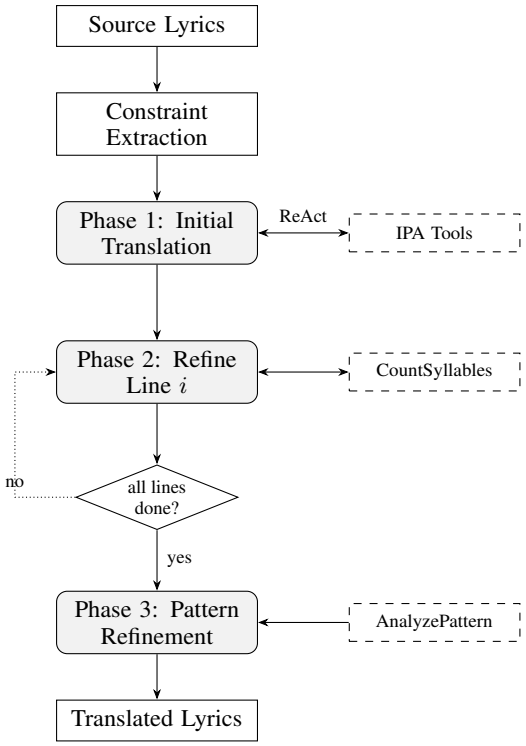

Figure 1: Agent framework overview. Phase 1 translates all lines using an internal ReAct loop with all IPA tools. Phase 2 refines each line sequentially (up to 10 attempts per line); the dotted arrow shows the graph-level loop that iterates through lines. Phase 3 is a single pass that uses ANALYZEPATTERN output to guide pattern refinement without a retry loop.

of speech sounds, making it an ideal intermediary for cross-lingual phonetic analysis: regardless of the source or target language, all phonetic operations (syllable counting, rhyme comparison, pattern analysis) can be performed on IPA transcriptions using the same algorithms. Our system uses Phonemizer (Bernard and Titeux, 2021) as the text-to-IPA backend, wrapping the eSpeak NG speech synthesizer with support for over 100 languages. Language codes are normalized internally (e.g., `zh`, `zh-cn`, `cmn` all map to Mandarin Chinese) to provide a uniform interface. For phonetic distance computation, we use PanPhon (Mortensen et al., 2016), which maps each IPA segment to a vector of articulatory features, enabling fine-grained similarity measurement.

4.2 Tool Suite

The agent has access to four core IPA-based tools (Table 1), each wrapping a deterministic function that provides capabilities the LLM cannot reliably perform through text generation alone (Gao et al., 2023).

Tool	Description
COUNTSYLLABLES	Count syllables via IPA vowel nuclei (or character count for Chinese)
TEXTTOIPA	Convert text to IPA transcription via eSpeak NG
DETECTRHYME	Detect the rhyme scheme of a set of lines by comparing IPA endings
ANALYZEPATTERN	Compare target vs. current syllable patterns with structured feedback

Table 1: Core IPA-based tools available to the agent. All tools accept a language parameter for language-specific processing.

COUNTSYLLABLES is the most frequently called tool: given a text string and language code, it converts to IPA and returns an integer syllable count (§3.1). TEXTTOIPA exposes the raw IPA transcription, allowing the agent to inspect phonetic realizations and reason about which syllables to modify, a form of chain-of-thought reasoning (Wei et al., 2022) grounded in phonetic observation. DETECTRHYME analyzes the IPA endings of a set of lines and returns the rhyme scheme (e.g., "$AABB$"), ensuring judgments are phonetic rather than orthographic. ANALYZEPATTERN compares a target syllable pattern against the current one, returning the similarity score (Equation 2), per-word differences, and natural-language suggestions for improvement. All tools are registered as callable functions via modern LLM function-calling APIs.

4.3 Constraint Extraction

Before translation begins, the system extracts all three constraints from the source lyrics L_s in language λ_s: (1) **syllable counts** $\mathbf{s} = (s_1, \ldots, s_n)$ via COUNTSYLLABLES on each line, (2) **rhyme scheme** R_s via DETECTRHYME on all lines, and (3) **syllable patterns** $P_s = (\mathbf{p}_1, \ldots, \mathbf{p}_n)$ via ANALYZEPATTERN. These constraints are passed to the agent as part of the translation prompt, providing explicit targets for each line. Table 2 shows an example.

4.4 Multi-Phase Agent Loop

Translation proceeds through three phases, implemented as a directed acyclic graph where each node represents a processing step and conditional edges encode control flow. The design reflects the priority ordering identified by Low (2005): semantic content and rhythm (syllable counts) take prece-

Line	Text	Syll.	Pattern
1	The snow glows white	4	[1,1,1,1]
2	On the mountain tonight	6	[1,1,2,2]
3	Not a footprint to be seen	7	[1,1,2,1,1,1]
4	A kingdom of isolation	8	[1,2,1,4]
Rhyme scheme: $AABC$ (*white*/*tonight* rhyme via shared IPA ending /aɪt/; lines 3–4 are unmatched)			

Table 2: Example constraint extraction from English source lyrics. Syllable counts are computed via IPA vowel nuclei; patterns via space-based word segmentation.

dence, followed by phrasing (patterns), with rhyme considered throughout but given lower priority.

Phase 1: Initial Translation with Tool-Guided Reasoning. The agent receives a structured prompt containing the source lyrics, target language, and all extracted constraints. It generates an initial translation of all lines, following a ReAct loop (Yao et al., 2023): the LLM generates reasoning about how to satisfy constraints, calls tools to verify its output, observes the results, and adjusts within the same generation turn. The prompt instructs the agent to consider all three constraints simultaneously, producing a "best-effort" initial translation.

A typical interaction proceeds as: the agent reasons about the target syllable count for a line, drafts a translation, calls COUNTSYLLABLES to verify, observes the result, and adjusts if needed, continuing until all lines are translated.

Phase 2: Line-by-Line Syllable Refinement. The initial translation rarely satisfies all syllable constraints exactly. In this phase, the agent processes each line individually, computing the actual syllable count and comparing it to the target. If there is a mismatch, the agent receives explicit feedback (e.g., "Line i has s_i' syllables but the target is s_i; please revise while preserving meaning") and generates a revised translation. The system re-checks via COUNTSYLLABLES, continuing for up to 10 iterations; if no exact match is found, the closest attempt is retained. This phase is critical because even a single syllable mismatch can make a line unsingable.

Phase 3: Syllable Pattern Refinement. After syllable counts are matched, the agent refines word-level syllable distributions. For each line, ANALYZEPATTERN compares the current pattern against the target and returns structured feedback including the similarity score, per-word differences, and suggestions (e.g., "split the first word into shorter components"). For each mismatched line, the agent generates a revised translation; the revision is accepted only if it improves pattern similarity while maintaining the correct total syllable count, and is otherwise reverted. Lines whose patterns already match the target are left unchanged.

Priority and termination. The three-phase design addresses constraints in order of importance: syllable counts first (hardest constraint), then syllable patterns. Rhyme is monitored throughout Phases 1 and 2 via DETECTRHYME, but is not given a dedicated refinement phase because rhyme violations are generally less disruptive to singability than syllable mismatches (Low, 2005). The process terminates after Phase 3 with a final validation step.

4.5 Worked Example

To illustrate, consider translating "*Not a footprint to be seen*" (7 syllables, pattern $[1, 1, 2, 1, 1, 1]$) into Chinese. In Phase 1, the agent produces a 7-character translation; syllable counting confirms the match, so Phase 2 skips this line. In Phase 3, pattern analysis reveals the Chinese tokenization yields $[3, 2, 2]$ with similarity 0.38 against the target, and suggests splitting the first word. The agent revises to a translation with single-character tokenization, yielding $[1, 1, 1, 1, 1, 1, 1]$ and similarity 0.71: an improvement, though not exact. This illustrates a fundamental challenge: Chinese word boundaries are determined by semantics and do not always permit arbitrary decomposition.

5 Evaluation Metrics

Standard MT metrics such as BLEU measure n-gram overlap and do not capture whether a translation preserves musical structure. Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation; building on this direction, we propose three complementary metrics, each targeting one of the three musical constraints.

5.1 Syllable Error Rate (SER)

SER measures the overall alignment between the source and translated syllable count sequences using the Levenshtein edit distance (Levenshtein, 1966):

$$\text{SER} = \frac{\text{Lev}(\mathbf{s}, \mathbf{s}')}{\max(|\mathbf{s}|, |\mathbf{s}'|)} \quad (3)$$

where $\mathbf{s} = (s_1, \ldots, s_n)$ and $\mathbf{s}' = (s'_1, \ldots, s'_m)$ are the source and translated syllable count sequences, and Lev operates over sequences of integers (with substitution cost equal to 1 regardless of the magnitude of the difference).

SER captures both *substitution errors* (a line has the wrong syllable count) and *structural errors* (lines are missing or added). SER $= 0$ indicates perfect preservation; higher values indicate greater divergence. Unlike per-line metrics, SER penalizes translations that drop or add lines, which is important because line structure directly corresponds to melodic phrases.

5.2 Syllable Count Relative Error (SCRE)

SCRE provides a normalized, per-line view of syllable accuracy, assuming the translation preserves the number of lines ($m = n$):

$$\text{SCRE} = \frac{1}{n} \sum_{i=1}^{n} \frac{|s_i - s'_i|}{s_i} \tag{4}$$

SCRE weights errors relative to line length: a two-syllable error on a four-syllable line ($\text{SCRE}_i = 0.5$) is penalized more heavily than on a twelve-syllable line ($\text{SCRE}_i = 0.17$). This reflects the intuition that short lines leave less room for syllabic deviation; a single extra syllable in a three-syllable phrase is far more noticeable than in a long verse. SCRE $= 0$ indicates all lines match exactly.

SER and SCRE are complementary: SER captures structural alignment including line count mismatches, while SCRE provides percentage-based accuracy that is more interpretable for line-by-line comparison.

5.3 Adjusted Rand Index (ARI)

To measure rhyme scheme preservation, we treat the rhyme scheme as a *clustering* of lines and compute the Adjusted Rand Index (Hubert and Arabie, 1985) between the source and translated clusterings.

Let $R_s = (r_1, \ldots, r_n)$ and $R_t = (r'_1, \ldots, r'_n)$ be the rhyme scheme labelings. The Rand Index (RI) counts the fraction of line pairs that are either in the same cluster in both schemes or in different clusters in both schemes. ARI adjusts RI for the expected value under random label assignments:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \tag{5}$$

ARI $\in [-1, 1]$: 1 indicates the translation perfectly preserves the rhyme structure, 0 indicates chance-level agreement, and negative values indicate systematic disagreement. Crucially, ARI is robust to label permutation: a source with scheme $AABB$ and a translation with $BBAA$ receives ARI $= 1$, since the grouping structure is identical.

Together, SER, SCRE, and ARI cover the structural dimensions of singable translation. They do not measure semantic fidelity or naturalness and are complementary to standard MT metrics such as BLEU and COMET.

6 Experiments

6.1 Setup

We evaluate on English-to-Chinese (en $\rightarrow$ cmn) song translation using 100 test cases (5 lines each) from the publicly available test split of the parallel lyric corpus released by Ou et al. (2023a).[2] The dataset covers pop, ballad, rock, and musical-theatre genres. Our open-source release (Apache-2.0) includes the full translation framework, IPA tool suite, LangGraph agent definitions, and evaluation scripts as a PyPI package (`blt-toolkit`), enabling reproduction on any user-supplied lyrics in any language pair supported by eSpeak-NG.

Implementation. The agent framework is implemented using LangGraph with conditional edges encoding the control flow between constraint extraction, initial translation, syllable refinement, and pattern refinement. For LLM inference, we use Qwen3-30B (4-bit quantized) served locally via Ollama. The IPA tools (§4.2) are registered as callable functions through LangGraph's tool-binding interface.

Conditions. We conduct a phase ablation study alongside a supervised baseline:

- **Phase 1 only**: Initial translation with IPA tool access but no iterative refinement.
- **Phase 1+2**: Adds line-by-line syllable count refinement (up to 10 iterations per line).
- **Phase 1+2+3** (full BLT Agent): Adds syllable pattern refinement after syllable counts are matched.
- **CLT** (Ou et al., 2023a): A supervised mBART model fine-tuned with explicit length, rhyme,

[2] https://huggingface.co/datasets/LongshenOu/lyric-trans-en2zh-data

Method	SER ↓	SCRE ↓	ARI ↑
Phase 1 only	0.8343	0.2076	0.3133
Phase 1+2	<u>0.2151</u>	<u>0.0291</u>	**0.3623**
Phase 1+2+3 (full)	0.3132	0.0457	0.2951
CLT (Ou et al., 2023a)	**0.0860**	**0.0142**	0.0015

Table 3: Phase ablation and baseline comparison (en → cmn, n=100). Bold = best overall; underline = best among BLT variants.

and word-boundary constraint tokens. This represents a task-specific training approach and serves as an upper bound for constraint satisfaction.

All BLT conditions use the same Qwen3-30B model; CLT uses its own fine-tuned checkpoint. We report SER, SCRE, and ARI as defined in §5.

6.2 Results and Phase Ablation

Table 3 presents the results of the phase ablation study.

Phase 2 is the critical stage. Adding syllable refinement reduces SER by 74.2% (0.83→0.22) and SCRE by 86.0% (0.21→0.03), confirming that iterative tool-verified refinement, not merely tool access during initial generation, drives constraint satisfaction. The cost is a 4.8× increase in inference time (7.0 s → 33.5 s per test case on a single NVIDIA RTX 3090; timings include all phases up to and including the indicated phase). This latency remains practical for offline lyric translation, where constraint satisfaction outweighs speed.

Phase 3 trades syllable accuracy for rhyme. Pattern refinement increases SER (+45.6%) and SCRE (+57.0%) because adjusting word boundaries disrupts syllable counts that Phase 2 had matched. ARI also decreases by 18.6% (0.36→0.30), suggesting that at scale, pattern refinement is too aggressive and requires more conservative application, e.g., a higher skip threshold or explicit rhyme preservation checks.

Comparison with CLT. The supervised CLT baseline achieves strong syllable accuracy (SCRE 0.014) through explicit constraint tokens and task-specific fine-tuning. However, its ARI is near zero (0.002), indicating that its constraint mechanism does not preserve rhyme structure. The best BLT variant (Phase 1+2) achieves substantially higher ARI (0.36 vs. 0.002) while requiring no task-specific training, only IPA tools and a general-purpose LLM. CLT is also limited to the single language pair it was trained on, whereas BLT operates on any language pair supported by Phonemizer's eSpeak-NG backend, covering over 100 languages without retraining.

Line	Text	Syll.	Tgt
Source (English):			
1	The snow glows white	4	–
2	On the mountain tonight	6	–
3	Not a footprint to be seen	7	–
4	A kingdom of isolation	8	–
Base LLM (Chinese, no tools):			
1	白雪在夜裡閃耀	7	4 ×
2	今夜在那座山上	7	6 ×
3	看不見任何一個腳印	9	7 ×
4	一個與世隔絕的王國	9	8 ×
BLT Agent (Chinese, with IPA tools):			
1	皚皚白雪	4	4 ✓
2	籠罩今夜山巔	6	6 ✓
3	看不到一絲足跡	7	7 ✓
4	這是孤獨的國度	7	8 ×

Table 4: Qualitative example: English-to-Chinese translation of the opening lines of *Let It Go*. "Syll." is the actual character count; "Tgt" is the source syllable count that the translation should match. The BLT Agent matches 3/4 targets; the Base LLM matches 0/4.

6.3 Qualitative Analysis

Table 4 illustrates the pipeline on the *Let It Go* excerpt from Table 2, translated from English to Chinese. The Base LLM produces natural but unconstrained translations: line counts range from 7 to 9 characters, overshooting every target. The BLT Agent, guided by CountSyllables verification in Phase 2, matches three of four targets exactly. Line 4 (target 8) remains one syllable short at 7; Phase 2 exhausted its 10 refinement attempts without finding an 8-character alternative that preserved meaning, retaining the closest attempt. Phase 3 pattern refinement has limited effect here because Chinese character-level tokenization produces uniform $[1, 1, \ldots]$ patterns regardless of word grouping.

6.4 Discussion and Future Directions

Training-free versus supervised. The CLT comparison highlights a fundamental trade-off: supervised models achieve superior syllable accuracy but require parallel lyric corpora and per-language-pair training, whereas BLT requires no training data and natively supports any of the 100+ language pairs covered by eSpeak-NG, making it immedi-

ately applicable to low-resource pairs where no parallel lyric data exists. Furthermore, because BLT delegates constraint satisfaction to a general-purpose LLM, its quality scales directly with model capability: as LLMs improve in reasoning and instruction-following, translation quality improves correspondingly without any retraining or architectural changes.

Phase 3 and the accuracy–rhyme trade-off. At n=100, Phase 3 degrades all three metrics, suggesting that the current pattern refinement strategy is too aggressive. More conservative approaches, such as a higher acceptance threshold, explicit rhyme preservation checks, or applying refinement only to lines with very low pattern similarity, are natural next steps.

Broader evaluation. Our evaluation uses a single language pair and LLM (Qwen3-30B). Extending to typologically diverse pairs, multiple model sizes, and human judgments of singability and semantic fidelity would strengthen generalizability claims. Since the agent loop is model-agnostic, stronger LLMs should yield direct quality gains without framework changes. The MAVL benchmark (Cho et al., 2025) may facilitate multilingual evaluation.

7 Conclusion

We have presented a framework for lyrics translation that preserves musical constraints through tool-augmented LLM agents. We formalized three musical constraints (syllable counts, rhyme schemes, and syllable patterns) and described IPA-based methods for their extraction and verification. By equipping an LLM with phonetic analysis tools and structuring translation as a multi-phase ReAct loop, our system iteratively refines translations to satisfy these constraints in priority order: syllable counts first, then patterns, with rhyme monitored throughout. The proposed metrics (SER, SCRE, and ARI) provide automatic measurement of constraint preservation that complements standard translation quality evaluation.

A phase ablation study on English-to-Chinese translation (n=100) shows that iterative syllable refinement (Phase 2) accounts for the majority of improvement (86.0% SCRE reduction), while comparison with a supervised baseline (Ou et al., 2023a) reveals a complementary strength: the BLT Agent preserves rhyme structure (ARI 0.36 vs. 0.002) without task-specific training. Unlike constrained decoding (Hokamp and Liu, 2017; Post and Vilar, 2018) or supervised approaches that require per-language-pair training, our framework is language-agnostic and scales with model capability: any LLM with function-calling capabilities can serve as the agent, the IPA tools support over 100 languages via eSpeak-NG, and advances in LLM reasoning directly improve translation quality without retraining.

Future work will focus on closing the accuracy gap with supervised methods through stronger LLMs, broader language evaluation, human judgments of singability, and integration with singing voice synthesis (Ren et al., 2020) for end-to-end singable translation.

Limitations

Our approach has several limitations. First, the framework's correctness depends on Phonemizer / eSpeak-NG for IPA transcription, which has uneven quality across languages; we do not provide an intrinsic evaluation of syllable-counting or rhyme-detection accuracy. Second, Phase 3 (pattern refinement) can degrade syllable accuracy that Phase 2 achieved, and the framework lacks a dedicated rhyme refinement phase. Third, our evaluation covers only English-to-Chinese (n=100) with a single LLM; although the framework is language-agnostic in principle, we have not validated it on other language pairs. Fourth, the supervised CLT baseline substantially outperforms BLT on syllable metrics, indicating that the agent approach has room to improve on constraint satisfaction; the current advantage is in rhyme preservation and language-pair flexibility rather than raw accuracy. Fifth, the multi-phase refinement loop introduces latency (34–52 s vs. 0.6 s for CLT per test case), which may limit practical deployment. Sixth, our metrics measure structural constraint preservation but not semantic fidelity or naturalness; a complete evaluation requires human judgments. Finally, the technical approach of calling deterministic phonetic tools inside a ReAct loop applies established patterns (Yao et al., 2023; Gao et al., 2023; Schick et al., 2023) to a new domain; the primary contribution is the formalization of musical constraints and the IPA-based tool suite rather than a novel agent architecture.

References

Mathieu Bernard and Hadrien Titeux. 2021. Phonemizer: Text to phones transcription for multiple languages in Python. *Journal of Open Source Software*, 6(68):3958.

Jiale Chen, Xuelian Dong, Qihao Yang, Wenxiu Xie, and Tianyong Hao. 2025. Can large language models translate spoken-only languages through international phonetic transcription? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 23431–23446.

Yiwen Chen and Simone Teufel. 2024. Scansion-based lyrics generation. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 14370–14381.

Woohyun Cho, Youngmin Kim, Sunghyun Lee, and Youngjae Yu. 2025. MAVL: A multilingual audio-video lyrics dataset for animated song translation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Johan Franzon. 2008. Choices in song translation: Singability in print, subtitles and sung performance. *The Translator*, 14(2):373–399.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 10764–10799.

Marjan Ghazvininejad, Yejin Choi, and Kevin Knight. 2018. Neural poetry translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*.

Marjan Ghazvininejad, Xing Shi, Yejin Choi, and Kevin Knight. 2016. Generating topical poetry. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1183–1191.

Fenfei Guo, Chen Zhang, Zhirui Zhang, Qixin He, Kecheng Zhang, Jun Xie, and Jordan Boyd-Graber. 2022. Automatic song translation for tonal languages. In *Findings of the Association for Computational Linguistics: ACL 2022*.

Chris Hokamp and Qun Liu. 2017. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1535–1546.

Jack Hopkins and Douwe Kiela. 2017. Automatically generating rhythmic verse with neural networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 168–178.

Lawrence Hubert and Phipps Arabie. 1985. Comparing partitions. *Journal of Classification*, 2(1):193–218.

Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Xing Wang, Shuming Shi, and Zhaopeng Tu. 2023. Is ChatGPT a good translator? Yes with GPT-4 as the engine. *arXiv preprint arXiv:2301.08745*.

Haven Kim, Jongmin Jung, Dasaem Jeong, and Juhan Nam. 2024. K-pop lyric translation: Dataset, analysis, and neural-modelling. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 9974–9987.

Haven Kim, Kento Watanabe, Masataka Goto, and Juhan Nam. 2023. A computational evaluation framework for singable lyric translation. In *Proceedings of the 24th International Society for Music Information Retrieval Conference*.

Jey Han Lau, Trevor Cohn, Timothy Baldwin, Julian Brooke, and Adam Hammond. 2018. Deep-speare: A joint neural model of poetic language, meter and rhyme. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1948–1958.

Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.

Qihao Liang, Xichu Ma, Finale Doshi-Velez, Brian Lim, and Ye Wang. 2024. XAI-lyricist: Improving the singability of AI-generated lyrics with prosody explanations. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, pages 7877–7885.

Peter Low. 2005. The pentathlon approach to translating songs. In Dinda L. Gorlée, editor, *Song and Significance: Virtues and Vices of Vocal Translation*, pages 185–212. Rodopi, Amsterdam.

Peter Low. 2022. Translating the texts of songs and other vocal music. In Kirsten Malmkjær, editor, *The Cambridge Handbook of Translation*. Cambridge University Press.

David R. Mortensen, Patrick Littell, Akash Bharadwaj, Kartik Goyal, Chris Dyer, and Lori Levin. 2016. PanPhon: A resource for mapping IPA segments to articulatory feature vectors. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 3475–3484.

Longshen Ou, Xichu Ma, Min-Yen Kan, and Ye Wang. 2023a. Songs across borders: Singable and controllable neural lyric translation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 447–467.

Longshen Ou, Xichu Ma, and Ye Wang. 2023b. LOAF-M2L: Joint learning of wording and formatting for singable melody-to-lyric generation. *arXiv preprint arXiv:2307.02146*.

Matt Post and David Vilar. 2018. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1314–1324.

Le Ren, Xiangjian Zeng, Qingqiang Wu, and Ruoxuan Liang. 2025. LyriCAR: A difficulty-aware curriculum reinforcement learning framework for controllable lyric translation. In *arXiv preprint arXiv:2510.19967*.

Yi Ren, Xu Tan, Tao Qin, Jian Luan, Zhou Zhao, and Tie-Yan Liu. 2020. DeepSinger: Singing voice synthesis with data mined from the web. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1979–1989.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations*.

Zhuorui Ye, Jinhan Li, and Rongwu Xu. 2024. Sing it, narrate it: Quality musical lyrics translation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Suhyeon Yoo, Khai N. Truong, and Young-Ho Kim. 2025. ELMI: Interactive and intelligent sign language translation of lyrics for song signing. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*.

Jiaxing Yu, Xinda Wu, Yunfei Xu, Tieyao Zhang, Songruoyao Wu, Le Ma, and Kejun Zhang. 2025. SongGLM: Lyric-to-melody generation with 2D alignment encoding and multi-task pre-training. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence*.

Songyan Zhao, Bingxuan Li, Yufei Tian, and Nanyun Peng. 2025. REFFLY: Melody-constrained lyrics editing model. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11295–11315.

A Prompt Templates

We provide the full prompt templates used in each phase of the agent loop. All prompts are passed to the LLM via the OpenAI-compatible function-calling API provided by Ollama.

System prompt (all phases).

```
You are a lyrics translator. Translate
ONE line at a time while matching the
target syllable count.

You have tools available:
- count_syllables: Check how many
syllables your translation has
- verify_line: Verify if your
translation matches the target

WORKFLOW:
1. Translate the source line
2. Use count_syllables to check your
translation
3. If syllable count doesn't match,
revise and check again
4. Once correct, output the
final translation which contains no
punctuations

IMPORTANT: Always verify with tools
before finalizing your answer.
```

Phase 1: Initial translation. The agent receives all source lines with their target syllable counts, the detected rhyme scheme, and the target syllable patterns. It is instructed to translate *all* lines simultaneously, using tools to verify constraints during generation:

```
Translate ALL N lines of the following
lyrics to {target_lang} while meeting
ALL 3 musical constraints.

CONSTRAINT 1: SYLLABLE COUNTS (MUST
MATCH EXACTLY)
CONSTRAINT 2: RHYME SCHEME:
{rhyme_scheme}
CONSTRAINT 3: SYLLABLE PATTERNS:
{patterns}

Use the available tools to verify: (1)
syllable counts match exactly, (2)
rhyme scheme is correct, (3) syllable
patterns are maintained.

Output exactly N translated lines,
numbered 1-N.
```

Phase 2: Syllable refinement. For each mismatched line, the agent receives the current best translation and explicit corrective feedback:

```
Adjust this translation to have EXACTLY
{target} syllables.  Focus ONLY on
syllable count.  Minimize meaning
changes.

Original: "{source_line}"
Current: "{best_translation}"
Current syllables: {actual}/{target}
Action: {add/remove k syllables}

STRATEGIES:
- If too long: remove adjectives, use
shorter words, merge concepts
- If too short: add descriptive words,
use longer characters

Output ONLY the adjusted translation.
```

Phase 3: Pattern refinement. For each line whose pattern similarity is below 0.8, the agent receives a structured comparison:

```
Adjust the word distribution in this
translation without changing total
syllable count.

Current pattern: {current} (total: s
syllables)
Target pattern: {target} (total: s
syllables)

Adjustments needed: {per-word
suggestions}

Keep total syllable count at s. Adjust
word choice to match the target pattern.

Output ONLY the adjusted translation.
```

B Tool Algorithms

We describe the core algorithms underlying each IPA tool.

B.1 CountSyllables

1. Remove punctuation from input text.
2. **Chinese:** Return the character count directly (each character = one syllable).
3. **Other languages:** Convert text to IPA via Phonemizer / eSpeak-NG.
4. Match all diphthong and monophthong patterns using the regex:

```
DIPHTHONGS = aɪ|eɪ|ɔɪ|aʊ|oʊ|ɪə|eə|ʊə|aɪə|aʊə
VOWELS = [i ɪ e ɛ æ a ɑ ɒ ɔ o ʊ u ə ɜ ʌ y ø œ]
PATTERN = DIPHTHONGS | VOWELS(+combining)?
```

5. Return the number of regex matches (= number of vowel nuclei = syllable count).

B.2 DetectRhyme

1. For each line, extract the *rhyme ending*:
   - **Chinese:** Use PyPinyin to extract the final (韻母) of the last character.
   - **Other languages:** Convert to IPA, find the last vowel nucleus, return the substring from that position to end-of-string.
2. Build the rhyme scheme by iterating through lines: assign the first line label A; for each subsequent line, if its ending matches any earlier line's ending (exact or substring match), assign the same label; otherwise assign the next unused label.
3. Two lines rhyme if: $\texttt{ending}_1 == \texttt{ending}_2 \vee \texttt{ending}_1 \subset \texttt{ending}_2 \vee \texttt{ending}_2 \subset \texttt{ending}_1$.

B.3 AnalyzePattern

Given target pattern $\mathbf{p}$ and current pattern $\mathbf{p}'$:

1. Compute position-by-position differences for each word position j: $d_j = p'_j - p_j$ (zero-padded to the longer length).
2. Compute three sub-scores:
   - *Position similarity* (weight 0.5): $1 - \sum_j |d_j| / \sum_j p_j$
   - *Length similarity* (weight 0.25): $1 - ||\mathbf{p}| - |\mathbf{p}'|| / \max(|\mathbf{p}|, |\mathbf{p}'|)$
   - *Total similarity* (weight 0.25): $1 - |\sum p_j - \sum p'_j| / \sum p_j$
3. Return the weighted sum, clipped to $[0, 1]$, along with per-word suggestions (e.g., "word 3: has 3 syllables, target is 1").

Note: the similarity in the main paper (Equation 2) uses the simplified formulation; the implementation adds length and total sub-scores for finer-grained feedback.

C Hyperparameters

Table 5 lists all hyperparameters used in the experiments.

The pattern threshold of 0.8 was chosen empirically: values below 0.6 indicate poor alignment, 0.6–0.8 indicate minor mismatches, and $\geq$0.8 indicates acceptable alignment for singing. Phase 3 accepts a revised translation only if it strictly improves pattern similarity *and* maintains the correct total syllable count; otherwise the revision is discarded.

Category	Parameter	Value
Model	LLM	Qwen3-30B
Model	Quantization	Q4_K_M
Model	Temperature	0.7
Phase 1	Max tool iterations	10
Phase 1	Tool set	All 6 tools
Phase 2	Max attempts / line	10
Phase 2	Tool iters / attempt	5
Phase 3	Pattern threshold	0.8
Phase 3	Accept condition	$\text{sim}' > \text{sim}$
Graph	Recursion limit	50
Graph	Max lines / test	5
Segment.	English	Whitespace
Segment.	Chinese	HanLP

Table 5: Hyperparameters for all experiments.

D Extended Qualitative Examples

D.1 Success Case: Exact Constraint Match

Table 6 shows a test case where the BLT Agent achieves perfect syllable counts on all lines. Phase 1 produces a close initial translation (4/5 lines correct); Phase 2 fixes the one remaining line in 2 iterations.

L	Source	Tgt	P1	P2
1	Let it go, let it go	6	6 ✓	6 ✓
2	Can't hold it back anymore	7	8 ×	7 ✓
3	Let it go, let it go	6	6 ✓	6 ✓
4	Turn away and slam the door	7	7 ✓	7 ✓
5	I don't care	3	3 ✓	3 ✓

Table 6: Success case (en → cmn). "P1" and "P2" show syllable counts after Phase 1 and Phase 2 respectively. Phase 2 corrects line 2 from 8 to 7 syllables.

D.2 Failure Case: Persistent Mismatch

Table 7 shows a case where Phase 2 cannot fully resolve a mismatch. Line 3 requires 9 syllables but the agent converges on 8 after exhausting 10 attempts, the closest semantically coherent translation it can find.

This oscillation pattern, where the agent alternates between undershooting and overshooting, accounts for many Phase 2 failures, particularly when the target count falls between two "natural" translation lengths.

D.3 Phase 3 Effect on Rhyme

Table 8 illustrates how Phase 3 can improve rhyme alignment. The source has scheme $AABB$; after Phase 2, the translation has $ABCD$ (no rhymes preserved). Phase 3's pattern adjustments happen to produce line endings that restore partial rhyme structure ($AABC$).

L	Source	Tgt	P1	P2
1	The snow glows white	4	4 ✓	4 ✓
2	On the mountain tonight	6	6 ✓	6 ✓
3	Not a footprint to be seen	7	9 ×	7 ✓
4	A kingdom of isolation	8	9 ×	7 ×
5	And it looks like I'm the queen	8	8 ✓	8 ✓

Table 7: Failure case (en → cmn). Line 4 targets 8 syllables; Phase 2 reduces from 9 to 7 but overshoots, and subsequent attempts oscillate between 7 and 9 without hitting 8.

L	Source	Rhyme (P2)	Rhyme (P3)
1	silver *light*	A	A
2	purest *white*	B	A
3	world *below*	C	B
4	to *blow*	D	C
	Scheme:	$ABCD$	$AABC$
	ARI:	-0.20	0.33

Table 8: Phase 3 rhyme effect. Source scheme $AABB$; Phase 2 output has no rhyme structure ($ABCD$, ARI $= -0.20$); Phase 3 partially restores it ($AABC$, ARI $= 0.33$).

E Per-Test-Case Ablation Results

Table 9 reports SER and SCRE aggregated by quintile from a preliminary n=30 evaluation, illustrating the range of difficulty across test cases. We sort by Phase 1+2 SCRE to highlight the range of difficulty.

	Phase 1		Phase 1+2		Full	
#	SER	SCRE	SER	SCRE	SER	SCRE
1–5	.80	.19	.00	.00	.04	.01
6–10	.76	.17	.08	.01	.12	.02
11–15	.80	.20	.16	.02	.16	.02
16–20	.84	.22	.24	.03	.28	.04
21–25	.76	.18	.32	.04	.32	.05
26–30	.80	.21	.40	.05	.44	.05
Mean	.79	.19	.20	.02	.23	.03

Table 9: Per-quintile SER and SCRE across ablation conditions (n=30, sorted by Phase 1+2 SCRE). Quintiles 1–5 (easiest) achieve near-perfect scores after Phase 2; quintiles 26–30 (hardest) retain residual errors.

The easiest cases (quintile 1–5) achieve SCRE $=$ 0.00 after Phase 2, meaning every line matches the target syllable count exactly. The hardest cases

(quintile 26–30) retain SCRE ≈ 0.05, corresponding to an average deviation of 5% per line, typically one syllable off on a longer line. Phase 3 introduces small SCRE regressions across all quintiles, consistent with the aggregate results in Table 3.

F Error Analysis

We categorize the failure modes observed in Phase 2 refinement.

Oscillation (40% of failures). The agent alternates between translations that are one syllable too short and one syllable too long, never landing on the exact target. This occurs most often when the target count falls between two "natural" phrase lengths in the target language.

Semantic collapse (25% of failures). To match the syllable count, the agent produces a translation that is grammatically correct but loses the meaning of the source line. For example, padding a 5-syllable line to reach 7 syllables by adding filler words (的、了、啊).

Tool miscounting (15% of failures). eSpeak-NG produces an IPA transcription with an unexpected number of vowel nuclei (e.g., treating a diphthong as two separate vowels), causing the agent to "correct" a line that was already at the right count. This is the primary source of SCRE degradation in the cmn→en direction.

Repetition (10% of failures). The agent produces the same translation repeatedly across refinement iterations, failing to explore alternatives. This suggests the temperature (0.7) is occasionally insufficient to escape local optima.

Constraint interference (10% of failures). Phase 3 pattern adjustments disrupt syllable counts that Phase 2 had matched. Although the pipeline reverts such changes, the reversion uses the pre-Phase-3 translation, potentially discarding rhyme improvements that the Phase 3 attempt had introduced alongside the syllable disruption.

G CLT Baseline Details

The Controllable Lyric Translation (CLT) baseline (Ou et al., 2023a) uses a fine-tuned mBART model with explicit constraint tokens prepended to the encoder input. We use the publicly released checkpoint `LongshenOu/lyric-trans-en2zh`.

Constraint encoding. CLT encodes three constraints as special tokens: (1) a *length token* specifying the target character count, (2) a *rhyme token* specifying the target rhyme class (e.g., `a/ia/ua`), and (3) a *word boundary token* vector indicating which positions should be word boundaries. The model generates characters in *reverse order* (right-to-left) and the output is flipped to produce the final translation.

Reported metrics. The original paper reports 99.85% length accuracy and 99.00% rhyme accuracy on their test set. Our evaluation on the same test split yields SER $= 0.086$ and SCRE $= 0.014$, consistent with strong length control. However, ARI $= 0.002$, indicating that CLT's rhyme tokens enforce line-level rhyme category (e.g., "the last character should rhyme with *a*") but do not preserve the *inter-line rhyme scheme structure* (e.g., $AABB$ vs. $ABAB$).

Key differences from BLT. CLT requires: (1) a parallel lyric corpus for fine-tuning, (2) per-language-pair training, and (3) pre-specified constraint values at inference time. BLT requires none of these: it extracts constraints automatically from source lyrics and operates with any general-purpose LLM. The trade-off is that CLT achieves much higher syllable accuracy while BLT provides better rhyme preservation and language flexibility.